

Parcial PRO2

1. Classes i orientació a objectes (Opcional)

- **Classes:** Converteixen structs en TADS amb privacitat (`public:` i `private:`) per assegurar consistència de dades.
- **Constructor:** Punts d'inici amb igual nom que la class. **Sobrecàrrega:** `Class()`, `Class(const Class& other)`.
- **Llista Inicialitzadors:** `: v1(0), v2(b) {}` Per referències i inicialitzar objectes membre de cop.
- **const methods:** `void get() const;` - Mètode que no modifica els atributs propis.
- **static methods:** Mètodes aplicables a nivell global des de la classe. `Classe::metode();`.
- **Immersions:** `inc_()`. **Atributs privats:** `int a_`.
- **Control de Fitxers (.hh/.cc) i Make:**
 - `.hh:` `1r) #ifndef XXXX_HH, 2n) #define XXXX_HH, 3r) class XXXX { ... }; 4t) #endif.`
 - `.cc:` `#include "XXXX.hh". S'aplica XXXX::XXXX(...) { a_ = a; }.`
 - `Makefile:` `XXXX.o: XXXX.hh XXXX.cc a sota g++ -c XXXX.cc.`
- **inline:** Elimina cost de crida funció inserint-ho en temps compilat a on toqui.
- **assert:** `#include "assert.hh".` Atura el programa automàticament si no compleix precondicions.

2. Piles i cues

- **Stack / Queue / PQ:** `#include <stack> / #include <queue>.` `pop()` sempre retorna void. Per obtenir el valor cal fer `top()` o `front()` abans del `pop()`.
- **Stack (LIFO):** `push(x), pop(), top(), empty(), size()`.
- **Queue (FIFO):** `push(x), pop(), front(), empty(), size()`.

3. Llistes, Vectors i Deques

- **list<T> / deque<T>:** `#include <list> / #include <deque>.`
 - **Mètodes comuns:** `push_back(x), push_front(x), pop_back(), pop_front(), insert(it, x), erase(it), back(), front()`.
 - *Diferència:* deque té operator[] ($O(1)$), list NO.
- **vector<T>:** `#include <vector>.`
 - **Mètodes:** `push_back(x), pop_back(), insert(it, x), erase(it), back(), front(), operator[]`.
- **Iteradors:**
 - `iterator:` Lectura i escriptura. `list<T>::iterator it`
 - `const_iterator:` Només lectura. `list<T>::const_iterator`
 - `reverse_iterator:` Invers, lectura i escriptura. `list<T>::reverse_iterator`
 - `const_reverse_iterator:` Invers, només lectura. `list<T>::const_reverse_iterator`
 - **Posicions:** `begin(), end(), rbegin(), rend()`.
 - **Ús:** `*it` (accés), `it++`, `it--`. Retorna `it = L.erase(it);`.

4. Arbres binaris (BinTree<T>)

- **BinTree<T>:** `#include "BinTree.hh".` Estructura recursiva immutable.
 - **Mètodes:** `value(), left(), right(), empty()`.

- **Recorreguts:** DFS (Pre/In/Postordre) i BFS (nivells).
- **Estratègies de Resolució:**
 1. **Casos base:** Comprovar `t.empty()` i, si cal, cas **fulla** (`left` i `right` buits).
 2. **Immersió:** Per si els arbres parlen entre ells (Ex: si tots nodes iguals, si totes fulles iguals, si suma de nodes fills = node actual, etc). Usar funció auxiliar (`nom__()`) amb paràmetre per **referència** (ex: `int& x`) per guardar estats.
 3. **Salt de fe:** Resol el node actual i assumeix que la recursivitat ja funciona per als fills.

5. Cues de prioritat i arbres generals

- **Heap<T>:** `#include "heap.hh".`
 - Mètodes: `push(x)`, `pop()`, `top()`, `empty()`, `size()`.
- **Tree<T>:** Arbre general (n-ari).
 - Mètodes: `value()`, `num_children()`, `child(i)`, `empty()`.

6. Diccionaris: map i set

- **set<T>:** `#include <set>.` **Conjunt ordenat** que no admet duplicats. Per usar `set` amb un `struct`, cal definir l' `operator<` (establir l'ordre).
 - Mètodes: `insert(x)`, `erase(x)`, `find(x)`, `empty()`, `size()`.
 - Iteradors: `begin()`, `end()`. S'accedeix al valor amb `*it`.
- **multiset<T>:** Igual que el `set`, però permet duplicats (útil per rànquings on dos elements poden empatar en tot).
- **map<K, V>:** `#include <map>.` Ordena automàticament per la **clau** (K). Elements interns: `pair<clau, valor>`.
 - Mètodes: `m[clau] = val`, `insert({clau, val})`, `erase(x)`, `find(clau)`, `empty()`, `size()`.
 - Iteradors: `begin()`, `end()`. S'accedeix com `it->first` (clau) i `it->second` (valor).

7. Rendiment i Complexitat (Resum)

Mètode	Stack/Queue	Heap (PQ)	Vector	Deque	List	Map/Set
push / insert	$O(1)$	$O(\log N)$	$O(1)^* / O(N)$	$O(1)^* / O(N)$	$O(1)$	$O(\log N)$
pop / erase	$O(1)$	$O(\log N)$	$O(1) / O(N)$	$O(1) / O(N)$	$O(1)$	$O(\log N)$
top / front / back	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	-
operator[]	-	-	$O(1)$	$O(1)$	-	$O(\log N)$
find	-	-	-	-	-	$O(\log N)$

**Cost amortitzat. insert(pos, x) i erase(pos) en list són $O(1)$ si ja tenim l'iterador.*